

Capítulo 2

Capa de Aplicación

Application
Transport
Network
Link
Physical

Capa de aplicación

Intención pedagógica:

- Este capítulo busca que el estudiante:
 - Comprenda cómo se diseñan y estructuran las aplicaciones de red desde la capa de aplicación.
 - Distinga arquitecturas, protocolos y tecnologías de implementación como piezas complementarias.
 - Desarrolle la capacidad de analizar decisiones de diseño (cliente/servidor, P2P, Web, sockets).
 - Conecte los conceptos con casos reales (BitTorrent, FTP, HTTP, Web moderna).
 - Adquiera criterios para evaluar ventajas, limitaciones y compromisos de cada enfoque.

Meta Formativa

- Que los estudiantes sean capaces de razonar como arquitectos de aplicaciones distribuidas, no solo como usuarios de APIs.

Capa de aplicación

- **Conocimientos previos asumidos:**
 - Conocimientos básicos de TCP y UDP
 - Conocimiento básico de sistemas de archivos
 - Nociones de direcciones IP

Capa de aplicación

Situación:

- Las aplicaciones distribuidas necesitan coordinarse, intercambiar información y entenderse entre sí, aun cuando:
 - Están en máquinas distintas
 - Usan sistemas operativos distintos
 - Están escritas en lenguajes distintos
 - Se ejecutan en redes heterogéneas
- La red por sí sola no resuelve cómo dos aplicaciones acuerdan qué decirse ni cómo interpretarlo.

Capa de aplicación

- La capa de aplicación define **cómo las aplicaciones usan la red** para intercambiar información y coordinarse.
- **La capa de aplicación responde a tres preguntas clave**
- **1. ¿Quién habla con quién?**
 - **Para esto se definen arquitecturas de aplicaciones de red**
 - Cliente/servidor
 - P2P
 - Híbridas
 - Tienen implicancias en escalabilidad, desempeño y robustez
- **2. ¿Qué mensajes se intercambian y qué significan?**
 - **Para esto se definen protocolos de aplicación**
 - Veremos qué es un protocolo
 - Partes constitutivas: Tipos de mensajes, sintaxis, semántica, reglas y estado
 - Derivaremos un ejemplo de diseño de un protocolo de transferencia de archivos (inspirado en FTP)

Capa de aplicación

- **3. ¿Cómo se implementa esa comunicación?**
 - **Para esto se usarán tecnologías de implementación**
 - Sockets (bajo nivel)
 - Web (alto nivel) como plataforma de aplicaciones
 - Abstracciones y compromisos
- La capa de aplicación es donde se diseña la lógica de interacción entre programas distribuidos.
- Todo lo demás (transporte, red, enlace) existe para que esta capa pueda funcionar.
- **Propósito**
 - Entender **cómo se construyen** las aplicaciones distribuidas y **por qué se diseñan así**.

Bloque 1: Arquitecturas de aplicaciones de red

- **Pregunta principal:** ¿Cómo organizamos una aplicación distribuida.
- La **arquitectura** es el esqueleto de una aplicación de red: define quién hace qué y cómo fluye la información.
- **Propósito:** Entender las formas fundamentales de organizar los nodos que participan en una aplicación de red y cómo esa organización condiciona:
 - La escalabilidad, la robustez, el desempeño, la distribución de responsabilidades.
- Elegir el tipo de arquitectura que se va a usar para desarrollar una aplicación de red es una decisión de diseño importante a tomar.

Bloque 1: Arquitecturas de aplicaciones de red

- ¿Qué es una arquitectura para aplicaciones de red?

Bloque 1: Arquitecturas de aplicaciones de red

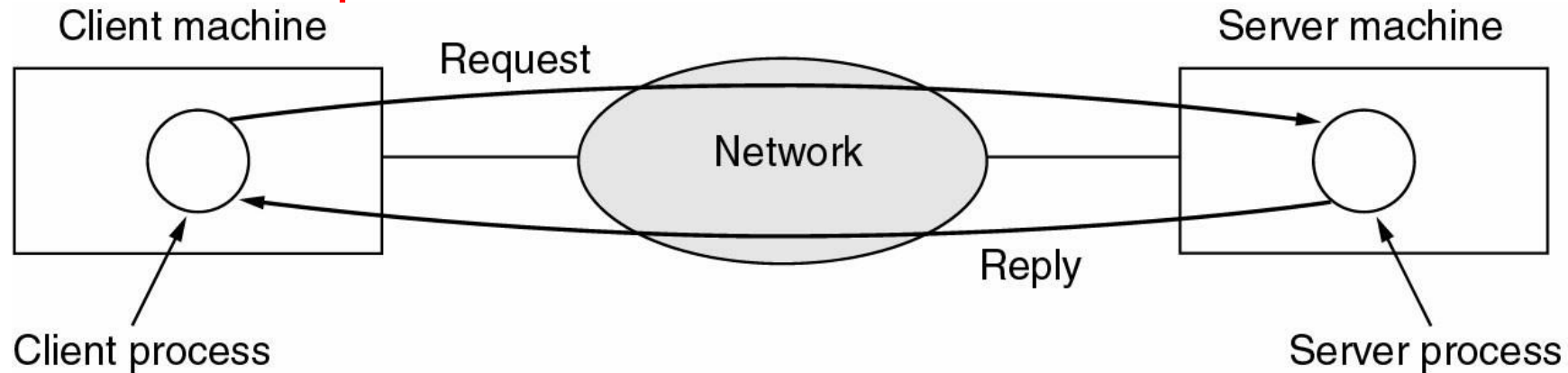
- **¿Qué es una arquitectura para aplicaciones de red?**
 - Tipos de nodos
 - Cada tipo de nodo tiene ciertas responsabilidades o se ocupa de ciertas tareas.
 - La comunicación entre nodos de dos tipos sigue un conjunto de reglas
- **Importancia de usar una arquitectura para aplicaciones de red**

Bloque 1: Arquitecturas de aplicaciones de red

- **¿Qué es una arquitectura para aplicaciones de red?**
 - Tipos de nodos
 - Cada tipo de nodo tiene ciertas responsabilidades o se ocupa de ciertas tareas.
 - La comunicación entre nodos de dos tipos sigue un conjunto de reglas
- **Importancia de usar una arquitectura para aplicaciones de red**
 - Sirve de guía para diseñar una aplicación de red.

Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas cliente-servidor



- **Basados en la figura indicar:** tipos de nodos, responsabilidades y reglas de comunicación

Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas cliente-servidor

- Tenemos nodos cliente y nodo servidor.
- El **servidor** es la componente dominante que controla el estado, coordina los clientes, almacena datos y es el único punto de acceso.
- El **cliente** hace pedidos de diferentes tipos y el servidor los procesa y responde.
- **Características**: control fuerte, simplicidad, pero poca escalabilidad sin infraestructura adicional.

Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas cliente-servidor

- **Sugerir cómo son las siguientes características de los nodos:** ubicación, direcciones IP, tiempo de vida.

Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas cliente-servidor

- **Para los servidores:**

- **Ubicación:** suele ser fija.
- **Direcciones IP:** suele no cambiar.
- **Tiempo de vida:** es común que sea largo.

- **Para los clientes:**

- **Ubicación:** pueden ser fijos o móviles.
- **Direcciones IP:** suelen ser dinámicas.
- **Tiempo de vida:** suele ser mas corta (usar el servidor y salir)

Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas cliente-servidor

- **Indicar los pasos que realiza una aplicación cliente-servidor que se apoya en UDP**
 - Asumir que tanto el pedido como la respuesta entra en un paquete (sino no tendría sentido usar UDP).
 - Considerar el caso que la respuesta no llega al cliente.

Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas cliente-servidor

- **Indicar los pasos que realiza una aplicación cliente-servidor que se apoya en UDP**
 1. El servidor elige un puerto y se queda esperando pedidos.
 2. El cliente construye un paquete UDP completo con toda la información necesaria (parámetros).
 3. El cliente envía un paquete UDP a la dirección IP y puerto del servidor.
 4. El servidor lee el paquete UDP completo, identifica qué cliente lo envió e interpreta el pedido.
 5. El servidor construye un mensaje de respuesta que debe caber en un solo paquete UDP.
 6. El servidor envía la respuesta al cliente.
 7. El cliente espera un paquete de respuesta por un tiempo limitado (timeout)
 8. Si ocurre el timeout, el cliente puede reenviar el pedido o abandonar según la lógica de la aplicación.

Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas cliente-servidor

- **Indicar los pasos que realiza una aplicación cliente-servidor que se apoya en TCP**
 - Asumir que el servidor se ejecuta antes que sus clientes.
 - Recordar que TCP funciona con creación y liberación de conexiones.
 - Asumir que el cliente puede hacer uno o más pedidos al servidor.

Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas cliente-servidor

- **Indicar los pasos que realiza una aplicación cliente-servidor que se apoya en TCP**
 1. El servidor elige un puerto y queda esperando conexiones.
 2. El cliente envía un pedido de conexión.
 3. Cuando el servidor recibe el pedido, acepta la conexión.
 4. El cliente envía un mensaje de solicitud a través de la conexión ya establecida.
 5. El servidor recibe el mensaje en el mismo orden en que fue enviado, lo interpreta y decide qué respuesta generar.
 6. El servidor envía un mensaje de respuesta por la misma conexión.
 7. Mientras la conexión siga abierta, el cliente puede enviar uno o varios pedidos adicionales sin necesidad de reconectar.
 8. Cuando el cliente termina de usar el servicio pide cerrar la conexión.
 9. El servidor confirma el cierre y ambos liberan la conexión.

Bloque 1: Arquitecturas de aplicaciones de red

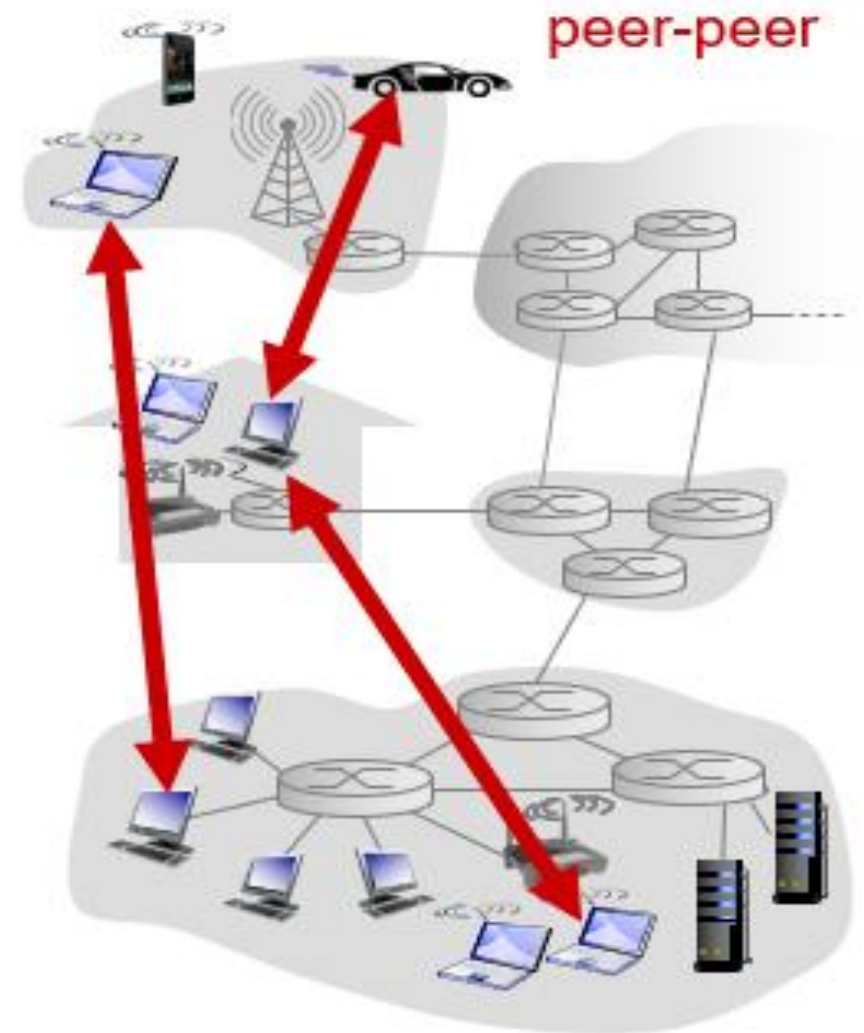
Arquitecturas compañero a compañero (P2P)

- La arquitectura cliente–servidor nos permitió entender un modelo claro y ordenado donde un servidor central coordina, almacena y responde a los pedidos de múltiples clientes.
- Pero ese orden tiene un costo: el servidor se convierte en un **punto único de carga, de falla y de control**.
 - A medida que las aplicaciones crecen en cantidad de usuarios, volumen de datos o necesidad de resiliencia, este modelo empieza a mostrar límites estructurales.
- Para superar esas restricciones surge otra forma de pensar la organización: distribuir el trabajo entre los propios nodos, permitir que colaboren directamente y eliminar la dependencia de un único punto central.
 - Este cambio de perspectiva nos lleva naturalmente al **modelo peer-to-peer**, donde los nodos dejan de ser meros consumidores y pasan a ser participantes activos en la red.

Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas compañero a compañero (P2P)

- ¿Qué es una arquitectura compañero a compañero? (indicar tipos de nodos – ver figura-, responsabilidades)



Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas compañero a compañero (P2P)

- ¿Qué es una arquitectura compañero a compañero?

- **Nodos:**

- Hosts llamados **compañeros**.
 - Ningún apoyo en servidores: **P2P puro**.
 - Mínimo apoyo en servidores: **predominantemente P2P**.

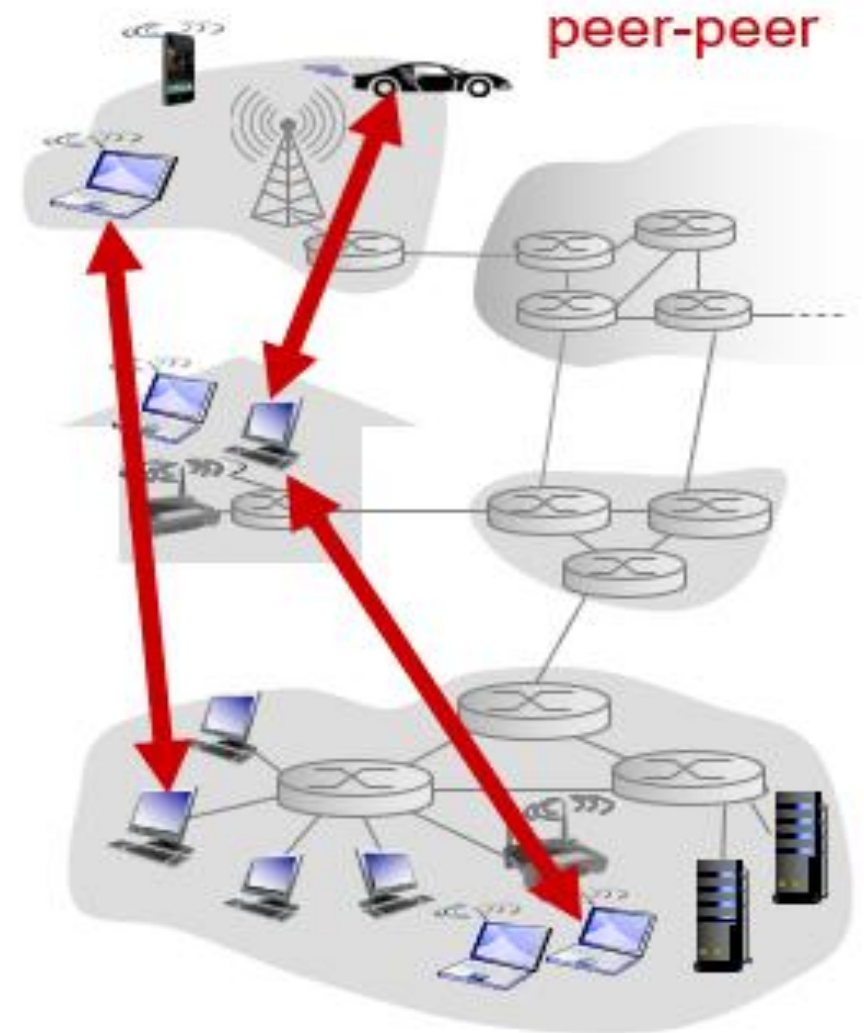
- **Responsabilidades:**

- **Compañeros:**

- Piden servicios de otros compañeros.
 - Proveen servicios en retorno a pedidos de otros compañeros.

- **Servidor mínimo:**

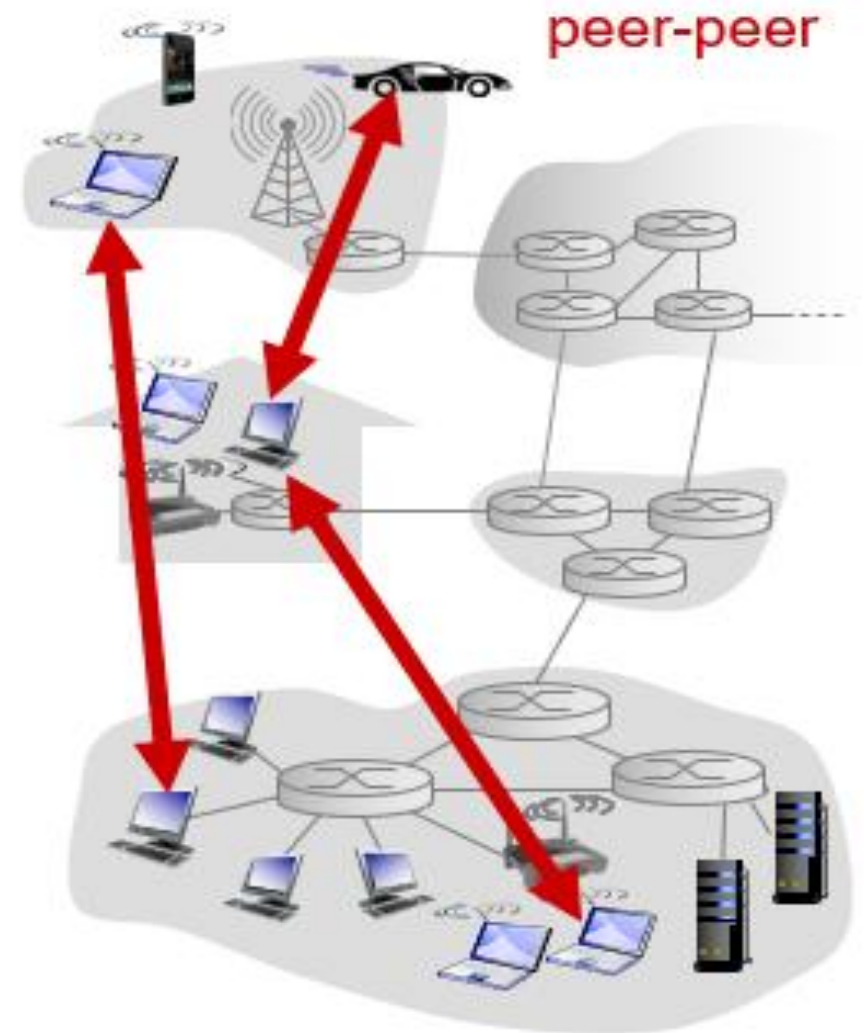
- Mantiene lista de nodos activos semipermanentes.
 - Entrega algunos compañeros iniciales a un compañero que recién entra.
 - Autenticación básica.



Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas compañero a compañero (P2P)

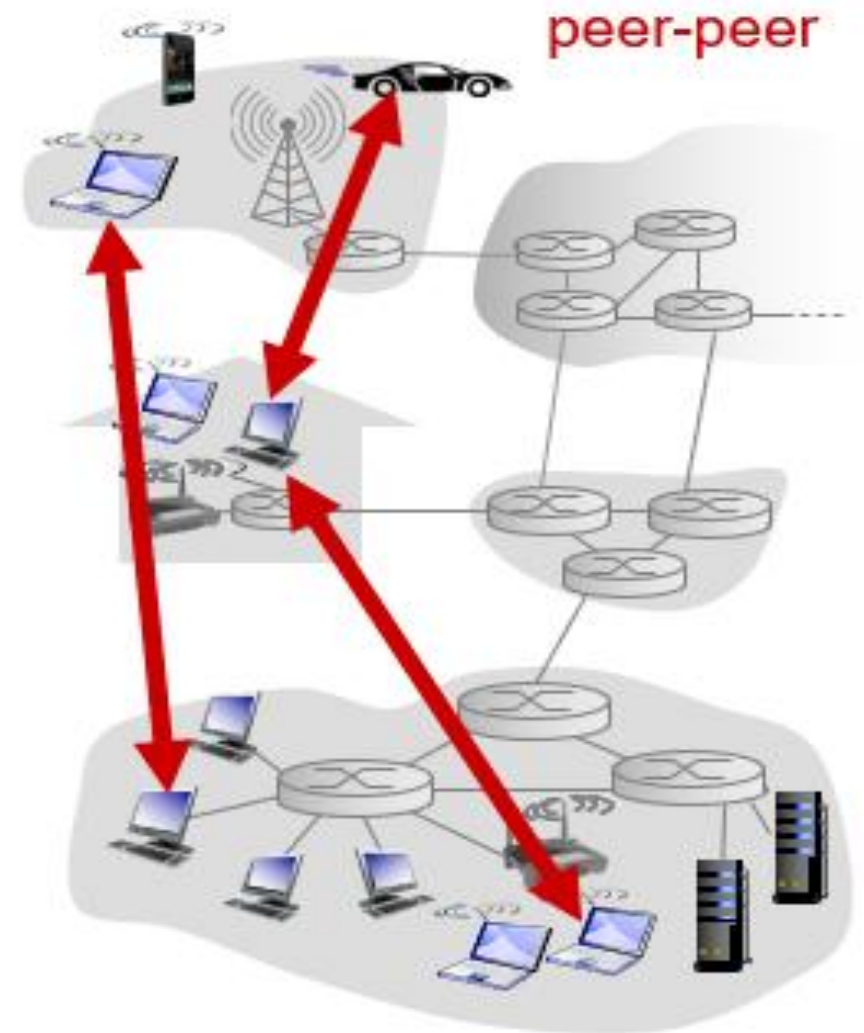
- Indicar las siguientes características de los compañeros: direcciones IP, tiempo de vida.



Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas compañero a compañero (P2P)

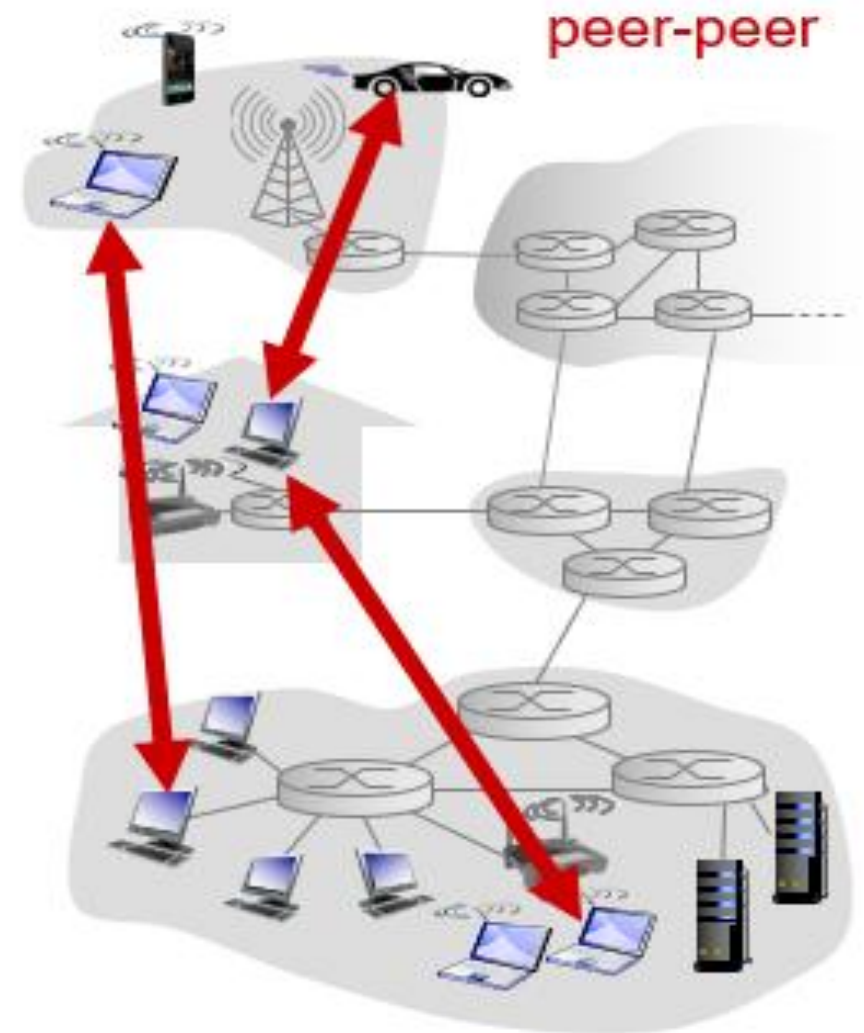
- **Indicar las siguientes características de los compañeros:** direcciones IP, tiempo de vida.
 - Los compañeros suelen tener IP dinámica y privada.
 - Los compañeros pueden entrar y salir en cualquier momento.
- **¿Qué efectos introduce la aparición de nuevos compañeros?**



Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas compañero a compañero (P2P)

- **Indicar las siguientes características de los compañeros:** direcciones IP, tiempo de vida.
 - Los compañeros suelen tener IP dinámica y privada.
 - Los compañeros pueden entrar y salir en cualquier momento.
- **¿Qué efectos introduce la aparición de nuevos compañeros?**
 - Más almacenamiento distribuido, más poder de cómputo.
 - Mayor redundancia y resiliencia (replicar datos, aumentar tolerancia a fallos de compañeros)
 - Aumento del trabajo de servidores mínimos que se usan.
 - Más fuentes y menor latencia promedio.

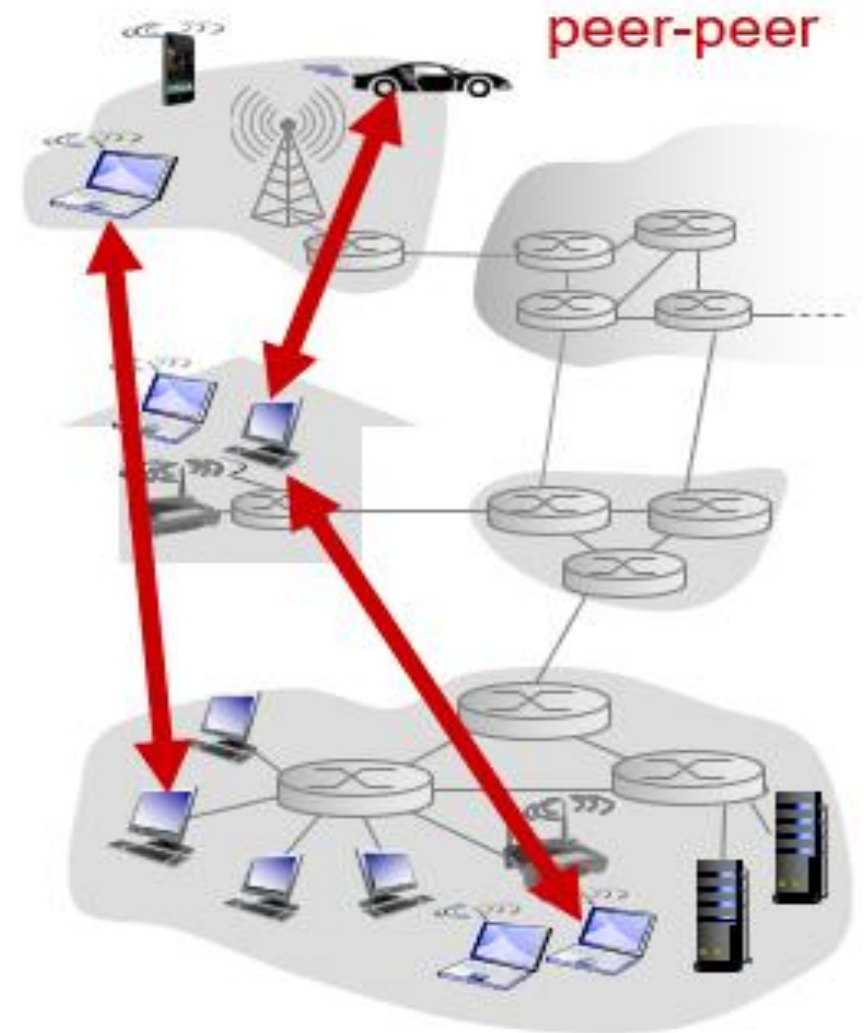


Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas compañero a compañero (P2P)

- **Ejemplos de aplicaciones P2P:**

- Distribución de archivos: BitTorrent, Emule.
- Telefonía por internet. Skype
- Bases de datos distribuidas
- Cálculos científicos distribuidos
- Transacciones entre pares con crypto monedas



Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **¿Qué problemas puede tener usar un servidor de archivos (arquitectura cliente/servidor)?** Explicar cada problema

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **¿Qué problemas puede tener usar un servidor de archivos (arquitectura cliente/servidor)?**
 - Sobrecarga del servidor,
 - Fallas del servidor,
 - Subida lenta/bajada rápida (p.ej. ADSL, redes de cable coaxial, 4G, 5G NSA)
- **¿Por qué resuelve los problemas anteriores usando arquitectura P2P?**

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **¿Qué problemas puede tener usar un servidor de archivos (arquitectura cliente/servidor)?**
 - Sobrecarga del servidor,
 - Fallas del servidor,
 - Subida lenta/bajada rápida (p.ej. ADSL, redes de cable coaxial, 4G, 5G NSA)
- **¿Por qué resuelve los problemas anteriores usando arquitectura P2P?**
 - La capacidad de la red aumenta con el número de compañeros. Porque cada compañero aporta recursos (ancho de banda, almacenamiento, rutas)
 - No hay un único nodo crítico, los datos están replicados o distribuidos en múltiples compañeros.
 - La red sigue funcionando aunque muchos nodos desaparezcan.
 - Los compañeros pueden descargar desde varios compañeros simultáneamente, esto busca reducir el problema que genera subida lenta y bajada rápida.

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **¿Cómo saber qué compañeros están activos?** (Sugerir alguna solución)

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **¿Cómo saber qué compañeros están activos?**
 - Tener una máquina con información de compañeros a la que se le puede preguntar.
 - En BitTorrent se llama **tracker**.
- **¿Qué tareas debe hacer un tracker?**

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **¿Cómo saber qué compañeros están activos?**

- Tener una máquina con información de compañeros a la que se le puede preguntar.
- En BitTorrent se llama **tracker**.

- **¿Qué tareas debe hacer un tracker?**

- A pedido de un compañero el tracker puede entregar la lista de los IP de los compañeros activos.
- Un compañero se registra en el tracker.
- Un compañero puede informar que se retira o avisar periódicamente al tracker que sigue ahí.

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- Asumimos que un archivo se divide en **trozos**.
- ¿Qué ventajas tiene esto?

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- Asumimos que un archivo se divide en **trozos**.
- **¿Qué ventajas tiene esto?**
 - Permite manejo mejor de velocidad de subida lenta y de bajada rápida.
 - Provee flexibilidad: un compañero no necesita tener un archivo entero para colaborar, solo algunos trozos.
- En un momento dado diferentes compañeros activos tienen diferentes subconjuntos de trozos de un archivo.

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **¿Qué trozos conviene pedir primero?**
 - Asumir que se puede pedir a un compañero que informe qué trozos tiene de un archivo.
 - Asumir que un compañero puede armar el cuadro de los trozos de un archivo que tienen los demás compañeros activos.

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **¿Qué trozos conviene pedir primero?**
 - Conviene pedir los trozos menos comunes primero.
 - **¿Por qué esto es importante?**

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **¿Qué trozos conviene pedir primero?**
 - Conviene pedir los trozos menos comunes primero.
 - **¿Por qué esto es importante?**
 - Para evitar que compañeros con trozos menos comunes se retiren y no pueda descargar esos trozos,
 - me viendo obligado a postergar la descarga del archivo.

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- Como la velocidad de subida es limitada un compañero no puede mandar trozos a todo el mundo que me pida.
 - Sino me puedo saturar y enviar trozos demasiado lento.
- Por lo tanto un compañero va a tener que decidir a quien enviar trozos primero.
- **Idear reglas de cómo elegir a quiénes enviar trozos primero**

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- Como la velocidad de subida es limitada un compañero no puede mandar trozos a todo el mundo que me pida.
 - Sino ese compañero se puede saturar y enviar trozos demasiado lento.
- Por lo tanto un compañero va a tener que decidir a quien enviar trozos primero.
- **Idear reglas de cómo elegir a quiénes enviar trozos primero**
 - **Regla tit-for-tat:** enviar trozos a aquellos X compañeros que le mandan trozos a una velocidad mayor.
 - Reevaluar periódicamente los X mejores compañeros.
 - **Criticar regla tit-for-tat y proponer una solución**

Bloque 1: Arquitecturas de aplicaciones de red

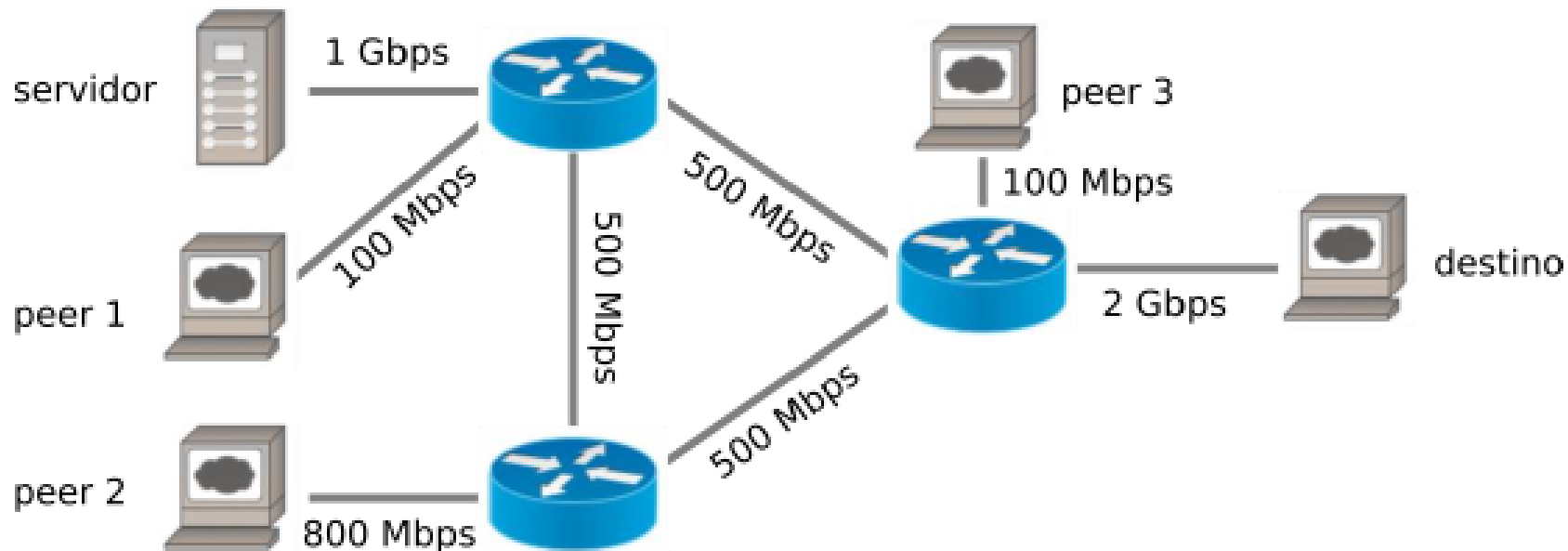
Distribución de archivos P2P

- Como la velocidad de subida es limitada un compañero no puede mandar trozos a todo el mundo que me pida.
 - Sino ese compañero se puede saturar y enviar trozos demasiado lento.
- Por lo tanto un compañero va a tener que decidir a quien enviar trozos primero.
- **Idear reglas de cómo elegir a quiénes enviar trozos primero**
 - **Regla tit-for-tat:** enviar trozos a aquellos X compañeros que le mandan trozos a una velocidad mayor.
 - Reevaluar periódicamente los X mejores compañeros.
 - **Criticar regla tit-for-tat y proponer una solución**
 - Un compañero puede enviar siempre trozos a los mismos compañeros y los otros compañeros se mueren de inanición.
 - Elegir aleatoriamente cada Y segundos otro compañero al cual enviar trozos.

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

Ejercicio 2: Basado en la siguiente red, determine y justifique el tiempo de descarga a *destino* de un archivo de 1.25 Gigabytes bajo el paradigma cliente/servidor y P2P (asuma que el archivo ya está distribuido entre los *peers*). Liste brevemente las ventajas y desventajas de cada paradigma. Ayuda: asuma que el enrutamiento es óptimo y que los enrutadores pueden dividir la carga del tráfico en varias interfaces.



Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **Ejercicio 2 - Continuación:** Haremos un análisis de mejor caso:
 - Vamos a asumir que los enrutadores están muy cerca entre sí, para no necesitar contar el tiempo de propagación.
 - Que el enrutamiento es óptimo significa:
 - No hay pérdida de paquetes.
 - Los paquetes llegan sin errores.
 - El tiempo de procesamiento no importa.

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **Ejercicio 2 - Continuación: Caso cliente-servidor**
 - Asumimos que solo el servidor manda datos a la mayor velocidad.
 - Los enrutadores no van a formar colas, van a mandar en cuanto reciben.
 - ¿cómo hacemos funcionar el enrutador conectado directamente al servidor?

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **Ejercicio 2 - Continuación: Caso cliente-servidor**

- Asumimos que solo el servidor manda datos a la mayor velocidad.
- Los enrutadores no van a formar colas, van a mandar en cuanto reciben.
- ¿cómo hacemos funcionar el enrutador conectado directamente al servidor?
- El enrutador divide el tráfico entre las dos líneas de salida, y el tráfico sigue por dos caminos.
- Como consecuencia el destino recibe datos a 1 Gbps.
- ¿Cuánto es 1,25 GB en bits?

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **Ejercicio 2 - Continuación: Caso cliente-servidor**

- Asumimos que solo el servidor manda datos a la mayor velocidad.
- Los enrutadores no van a formar colas, van a mandar en cuanto reciben.
- **¿cómo hacemos funcionar el enrutador conectado directamente al servidor?**
- El enrutador divide el tráfico entre las dos líneas de salida, y el tráfico sigue por dos caminos.
- Como consecuencia el destino recibe datos a 1 Gbps.
- **¿Cuánto es 1,25 GB en bits?**
- $1,25 \text{ GB} = 1,25 * 2^{30} * 8 \text{ b} = 10 * 2^{30} \text{ b} = 10 * 1037 \text{ millones b} = 10,37 \text{ Gb.}$
- Luego el tiempo de recepción del archivo es de 10,37 segundos.

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **Ejercicio 2 - Continuación: Caso P2P**
 - Asumir que servidor se comporta como un compañero más.
 - ¿Cuánto es la máxima cantidad de Gbps que le puede llegar al enrutador de destino considerando los enlaces que llegan al enrutador más a la derecha?

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **Ejercicio 2 - Continuación: Caso P2P**

- Asumir que servidor se comporta como un compañero más.
- ¿Cuánto es la máxima cantidad de Gbps que le puede llegar al enrutador de destino considerando los enlaces que llegan al enrutador más a la derecha?
- Contando todas las líneas de entrada al enrutador, puede mandar a 1,1 Gbps.
- ¿Suponiendo que llegan datos a esa velocidad, cuantos segundos toma descargar el archivo completo?

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **Ejercicio 2 - Continuación: Caso P2P**

- Asumir que servidor se comporta como un compañero más.
- ¿Cuánto es la máxima cantidad de Gbps que le puede llegar al enrutador de destino considerando los enlaces que llegan al enrutador más a la derecha?
- Contando todas las líneas de entrada al enrutador, puede mandar a 1,1 Gbps.
- ¿Suponiendo que llegan datos a esa velocidad, cuantos segundos toma descargar el archivo completo?
- Ya vimos antes que 10,37 Gb es el tamaño del archivo
- 1 seg ----- 1,1 Gb
- X ----- 10,37 Gb
- $X = 9,42$ segundos.

Bloque 1: Arquitecturas de aplicaciones de red

Distribución de archivos P2P

- **Ejercicio 2 - Continuación: Caso P2P**

- ¿Cómo puede llegar 1,1 Gbps al enrutador más a la derecha? Señalar alternativas.
- Hay muchas posibilidades para lograrlo: el servidor manda 500 Mbps y peer 2 manda a 500 Mbps y el peer 3 manda a 100 Mbps.
- Dar otros ejemplo ustedes.

Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas híbridas

- En una arquitectura predominantemente P2P, la mayor parte del trabajo lo realizan los propios nodos: comparten datos entre sí, colaboran directamente y reparten la carga de manera bastante equilibrada.
 - Los servidores que existen en este modelo suelen ser muy livianos: solo ayudan a que los nodos se encuentren o sepan quién está disponible, pero no intervienen en la parte pesada del sistema.
 - Este enfoque funciona bien mientras la red es relativamente simple y los nodos pueden coordinarse sin demasiadas dificultades.
- Sin embargo, cuando la aplicación crece, aparecen necesidades más complejas: algunos nodos necesitan ayuda para conectarse, otros requieren asistencia para iniciar la comunicación, y a veces hace falta mantener cierta información organizada para que la red funcione de manera fluida.
 - En estos casos, los servidores livianos ya no alcanzan.
 - Se vuelve necesario incorporar servidores más capaces, que no solo ayuden a encontrarse, sino que también colaboren en tareas de organización, coordinación y manejo de situaciones más complejas.
- Este paso natural da lugar a las arquitecturas híbridas, donde los servidores tienen un rol más activo —aunque sin llegar a controlar todo como en cliente-servidor— y los nodos siguen participando en la transferencia y el procesamiento distribuido.
 - Es un punto intermedio que combina la flexibilidad del P2P con un nivel de apoyo central que hace que la red sea más estable, más fácil de usar y más confiable.

Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas híbridas

- Una **arquitectura híbrida** combina elementos del modelo cliente/servidor y del modelo P2P para aprovechar las ventajas de ambos.
 - En lugar de depender completamente de un servidor central o distribuir toda la responsabilidad entre pares,
 - asignan algunas funciones de **coordinación** a servidores y las tareas de **transferencia o procesamiento** a los nodos pares.
- **Surgen como respuesta a dos tensiones clásicas:**
 - La necesidad de escalabilidad y distribución de carga (fortalezas del P2P).
 - La necesidad de coordinación, descubrimiento de nodos y control (fortalezas del cliente/servidor).

Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas híbridas

- **¿Qué significa coordinación?** (pensar en qué hace una persona con rol de coordinador)

Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas híbridas

- **Asuntos contemplados por la coordinación:**
 - **Asignación de roles:** decidir quién hace qué.
 - **Sincronización:** asegurar que las acciones ocurran en el orden correcto.
 - **Consistencia del estado:** mantener información coherente entre nodos.
 - **Manejo de conflictos:** resolver situaciones donde dos nodos quieren hacer cosas incompatibles (lo que genera resultados incorrectos, inconsistentes o desordenados).
- Sin coordinación, la red sería un conjunto de nodos aislados sin capacidad de colaborar.

Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas híbridas

- **Las arquitecturas híbridas buscan un equilibrio:** un sistema que pueda crecer sin colapsar, pero que mantenga puntos de referencia estables para organizar la interacción entre nodos.
- Aquí los servidores son **más pesados**.
- **Sugerir funciones que pueden cumplir distintos tipos de nodos.**

Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas híbridas

- Los **servidores** pueden:
 - hacer señalización compleja
 - descubrimiento (¿quién está disponible?),
 - negociación (los nodos acuerdan cómo comunicarse – protocolo a usar y versión, qué capacidades tiene cada uno),
 - establecimiento de sesión (intercambiar identificadores, intercambiar claves, preparar el canal),
 - mantenimiento de parte del estado (quién está conectado, quién está disponible),
 - participar parcialmente en la distribución de contenido,
 - autenticación,
 - Indexación,
 - coordinación.
- Los **compañeros** realizan la transferencia de datos o el procesamiento distribuido.
 - Esto permite reducir la carga del servidor sin perder orden ni coherencia en el sistema.

Bloque 1: Arquitecturas de aplicaciones de red

Arquitecturas híbridas

- **Características:**

- Mejor control y seguridad
- Mejor experiencia del usuario
- Menos descentralización real
- Más infraestructura necesaria

- **Ejemplos de arquitecturas híbridas:**

- Skype clásico.
- Juegos online modernos (matchmaking centralizado + comunicación entre jugadores).
- CDNs (las vimos en la introducción: coordinación centralizada y nodos distribuidos de entrega).

Bloque 1: Arquitecturas de aplicaciones de red

Comparación de los tipos de arquitectura

Criterio	Cliente-servidor	Predominantemente P2P	Híbrida
Escalabilidad	Limitada por capacidad del servidor	Muy alta: la carga se reparte entre los nodos.	Alta: los servidores ayudan, pero los pares trabajan.
Robustez	Baja: si el servidor falla, el servicio se cae.	Alta: la red funciona cuando algunos nodos se desconectan.	Media-Alta: los servidores ayudan a estabilizar, pero no son únicos puntos de falla.
Control	Centralizado: el servidor decide y organiza todo.	Distribuido: cada nodo participa sin jefe central.	Mixto: algunas cosas se controlan desde servidores, otras desde los pares.
Costos de infraestructura	Altos: el servidor debe ser potente y estar siempre disponible.	Bajo: la mayor parte lo hacen los nodos.	Medios: se necesitan servidores mas capaces que en P2P, pero no tan críticos como en cliente-servidor.
Rendimiento bajo carga	Puede degradarse si muchos clientes acceden al mismo tiempo.	Mejora con más nodos: cuantos mas participantes, más capacidad.	Suele ser estable: los servidores ordenan y los pares distribuyen la carga.

Bloque 1: Arquitecturas de aplicaciones de red

- **Recapitulación:**

- En este bloque vimos que las arquitecturas definen la manera en que se organiza una aplicación distribuida.
 - El modelo cliente/servidor asigna roles fijos y centraliza el control, lo que simplifica el diseño pero introduce límites de escalabilidad.
 - En contraste, las arquitecturas P2P distribuyen responsabilidades entre nodos equivalentes, lo que mejora la robustez y reparte la carga, aunque exige mecanismos adicionales para coordinar a los compañeros.
 - También mencionamos arquitecturas híbridas que combinan coordinación central con distribución, buscando un equilibrio entre control y escalabilidad.
- Comprendimos que la arquitectura elegida condiciona el comportamiento global del sistema:
 - determina cuán escalable puede ser, qué tan robusto es ante fallas, cómo se distribuyen los costos operativos y qué responsabilidades recaen en cada nodo.
 - La arquitectura no es un detalle técnico, sino una decisión estructural que define el esqueleto de la aplicación.
- **Idea final:** elegir una arquitectura es decidir cómo fluye la información y cómo se reparte el trabajo dentro de la aplicación.

Bloque 2: Protocolos de aplicaciones de red

- Las arquitecturas nos permitieron entender cómo se organiza una aplicación distribuida y qué roles asumen los nodos dentro de ella. Pero una arquitectura, por sí sola, no alcanza para que los nodos puedan colaborar: saber quién hace qué no explica cómo se comunican ni qué reglas siguen para coordinarse.
- Para que una aplicación distribuida funcione, los nodos necesitan un lenguaje común, un conjunto de mensajes y normas que les permita interactuar de manera coherente.
- Ese puente entre la estructura y la comunicación es justamente el territorio de los protocolos de aplicación, que exploraremos en este bloque.

Bloque 2: Protocolos de aplicaciones de red

- **Pregunta principal:** ¿Cómo se entienden los nodos de una aplicación de red entre sí?
- Un **protocolo** es el lenguaje de la aplicación distribuida: sin él, no hay coordinación posible.
- **Propósito:** Comprender cómo dos aplicaciones acuerdan qué mensajes intercambiar, cómo interpretarlos y cuándo enviarlos, para lograr una interacción coherente y predecible.

Bloque 2: Protocolos de aplicaciones de red

- Para construir aplicaciones de red definir protocolos.
- **Repaso:** ¿Qué es un protocolo?
- **¿Qué cosas definir en un protocolo de aplicación de red?**

Bloque 2: Protocolos de aplicaciones de red

- Para construir aplicaciones de red definir protocolos.
- **Repaso:** ¿Qué es un protocolo?
- **¿Qué cosas definir en un protocolo de aplicación de red?**
 - Tipos de mensajes
 - Sintaxis del mensaje
 - Semántica del mensaje
 - Reglas
 - Estado de la aplicación

Bloque 2: Protocolos de aplicaciones de red

- La aplicación necesita recordar cosas para interpretar correctamente los mensajes
 - En una red, los nodos no comparten memoria RAM.
 - Para coordinarse, cada aplicación debe recordar información relevante sobre la interacción en curso.
- El **estado** es esa información que la aplicación mantiene entre mensajes

Bloque 2: Protocolos de aplicaciones de red

- Ejemplos típicos de estado:

- Quién es el **otro extremo** (identidad, autenticación)
- En qué **parte del diálogo** estamos (fase del protocolo)
- Qué **recursos** estamos usando (p.ej. directorio actual, archivo abierto, etc.)
- Qué se envió o recibió antes (para interpretar lo que viene después)
- **Configuraciones activas**: son decisiones o parámetros que la aplicación negocia o establece durante la interacción.
 - Son opciones operativas que determinan cómo se va a comunicar la aplicación a partir de ese momento.
 - Ejemplos:
 - **opciones negociadas al inicio de sesión**: versión del protocolo, algoritmos de compresión, algoritmos de cifrado, capacidades soportadas por cada extremo.
 - **Modo operativo elegido** por el usuario o protocolo: por ejemplo: modo binario o ASCII, Modo de autenticación (simple, token, challenge-response)

Bloque 2: Protocolos de aplicaciones de red

¿Por qué es necesario el estado?

- Porque sin estado:
 - cada mensaje sería ambiguo
 - no habría forma de interpretar acciones en contexto
 - no se podrían mantener sesiones, transferencias ni operaciones compuestas
 - La aplicación no podría coordinarse con otras
- El estado es la memoria que permite que una aplicación distribuida tenga continuidad, coherencia y sentido a lo largo de una interacción.

Bloque 2: Protocolos de aplicaciones de red

Protocolo de transferencia de archivos

- **Objetivo:** Definir un protocolo de transferencia de archivos usando arquitectura cliente/servidor.
- **¿Qué tipos de mensajes puede tener el protocolo?** (**ayuda:** asumir que se parece a manejar un sistema de archivos con comandos)

Bloque 2: Protocolos de aplicaciones de red

Protocolo de transferencia de archivos

- **Objetivo:** Definir un protocolo de transferencia de archivos usando arquitectura cliente/servidor.
- **¿Qué tipos de mensajes puede tener el protocolo?**
 - Mensajes de comandos
 - Mensajes de feedback (de respuesta a comandos)
 - Mensajes con información (directorios, archivo)
- **¿Qué tipos de comandos usar?**

Bloque 2: Protocolos de aplicaciones de red

Protocolo de transferencia de archivos

- **Objetivo:** Definir un protocolo de transferencia de archivos usando arquitectura cliente/servidor.
- **¿Qué tipos de mensajes puede tener el protocolo?**
 - Mensajes de comandos
 - Mensajes de feedback (de respuesta a comandos)
 - Mensajes con información (directorios, archivo)
- **¿Qué tipos comandos a usar?**
 - Para autenticarse
 - Para obtener lista de archivos en folder
 - Para retornar archivo
 - Para subir archivo

Bloque 2: Protocolos de aplicaciones de red

Protocolo de transferencia de archivos

Sintaxis de comandos

- **USER *username***
- **PASS *password***
- **LIST** return list of file in current directory
- **RETR *filename*** retrieves (gets) file
- **STOR *filename*** stores (puts) file onto remote host

Sintaxis de Mensajes de respuesta

- ❖ Código de estatus y frase

Ejemplos de Mensajes de respuesta

- 331 Username OK, password required
- 125 data connection already open; transfer starting
- 425 Can't open data connection
- 452 Error writing file

Bloque 2: Protocolos de aplicaciones de red

Protocolo de transferencia de archivos

- ¿Qué les parece la idea de tener una sola conexión persistente entre el cliente y el servidor?

Bloque 2: Protocolos de aplicaciones de red

Protocolo de transferencia de archivos

- ¿Qué les parece la idea de tener una sola conexión persistente entre el cliente y el servidor?
 - Supongamos tenemos:
 - RETR archivo.txt
 - `125 data connection already open; transfer starting`
 - <datos del archivo>
 - LIST
 - <datos del listado>
 - Surge el problema de donde termina el archivo (no hay delimitadores seguros)
 - Surge el problema de separar el feedback del archivo.

Bloque 2: Protocolos de aplicaciones de red

Protocolo de transferencia de archivos

- ¿Cómo resolver los problemas anteriores usando dos conexiones separadas?

Bloque 2: Protocolos de aplicaciones de red

Protocolo de transferencia de archivos

- **¿Cómo resolver los problemas anteriores usando dos conexiones separadas?**
 - Usar un puerto para comandos y otro para datos evita el segundo problema.
 - Abrir una conexión solo para enviar un archivo o contenido de folder y luego cerrarla resuelve el primer problema.

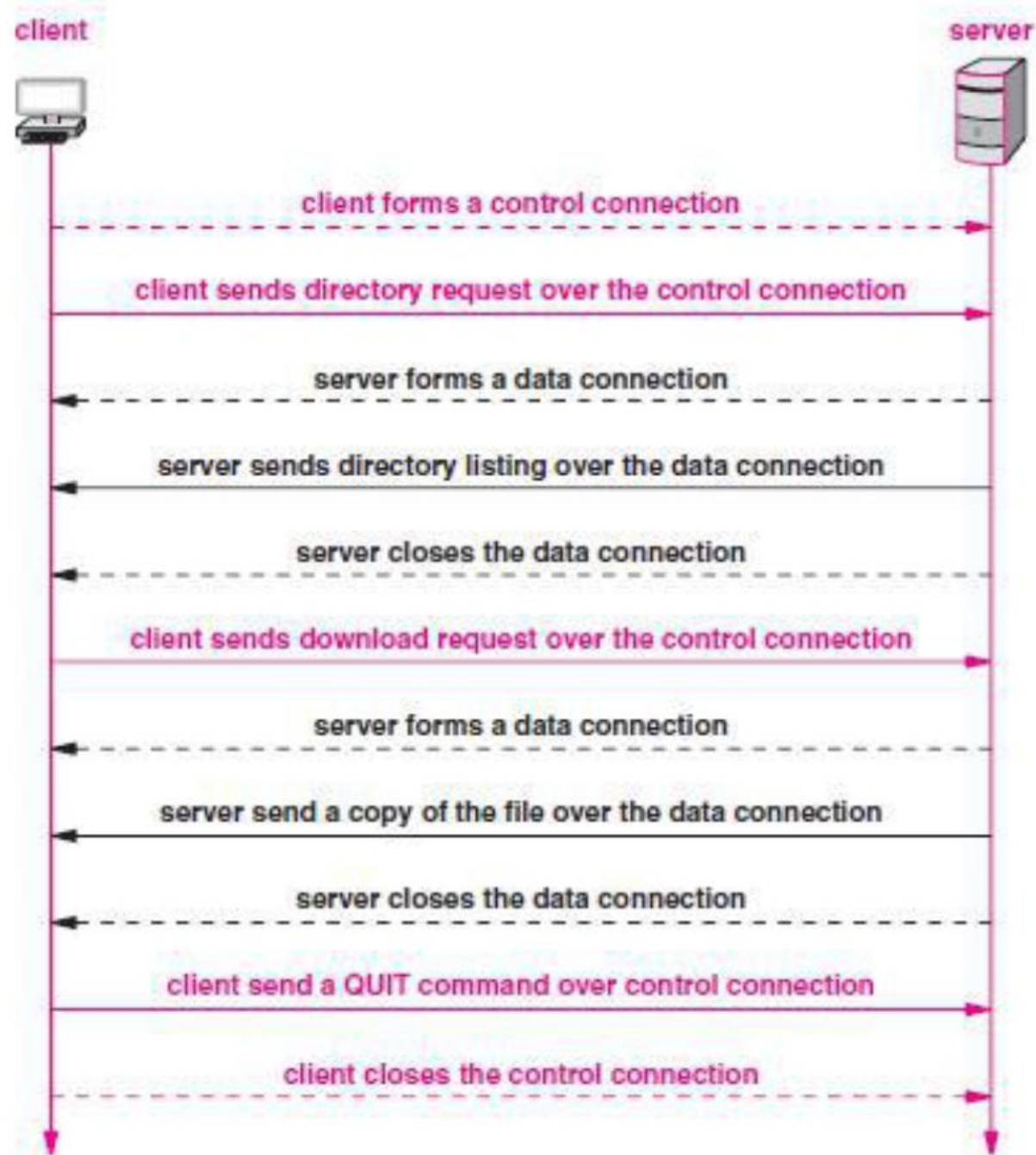
Bloque 2: Protocolos de aplicaciones de red

Protocolo de transferencia de archivos

¿Cuáles son las reglas para transferir un archivo?

- Asumir que primero se reciben los contenidos de un folder y que luego se recibe un archivo.
- Usar que hay conexiones de control y de datos.

Bloque 2:
Protocolos de
aplicaciones de
red
Protocolo de
transferencia de
archivos



Bloque 2: Protocolos de aplicaciones de red

Protocolo de transferencia de archivos

- ¿Qué información de estado puede manejar el protocolo?

Bloque 2: Protocolos de aplicaciones de red

Protocolo de transferencia de archivos

- ¿Qué información de estado puede manejar el protocolo?
 - directorio corriente,
 - autenticación previa.

Bloque 2: Protocolos de aplicaciones de red

Recapitulación:

- Un protocolo es un acuerdo entre aplicaciones que define cómo se comunican.
- Un protocolo especifica qué tipos de mensajes existen, cómo se estructuran, qué significan y en qué orden deben intercambiarse.
 - También define qué estado mantiene la aplicación para interpretar correctamente esos mensajes.
- Sin un protocolo claro, dos aplicaciones no podrían coordinarse ni interpretar sus acciones mutuamente.
- Discutimos cómo el diseño del protocolo refleja decisiones arquitectónicas: qué se envía, cuándo se envía y cómo se interpreta.
 - El protocolo es la manifestación concreta de la lógica de interacción definida por la arquitectura.

Bloque 3: Tecnologías de implementación

- Vimos que un protocolo define qué mensajes existen, cómo se interpretan y en qué orden se intercambian. Pero un protocolo es una idea abstracta: describe la lógica de interacción, no la forma concreta de implementarla.
- Para que un protocolo cobre vida, necesitamos herramientas que permitan enviar y recibir datos, gestionar conexiones y materializar la comunicación entre nodos.
- Es aquí donde entran en juego las tecnologías de implementación, que determinan el nivel de control, la complejidad y las capacidades reales de una aplicación distribuida.
- En este bloque veremos cómo estas tecnologías —desde sockets hasta la Web— condicionan y habilitan distintas formas de construir sistemas en red.

Bloque 3: Tecnologías de implementación

- **Pregunta principal:** ¿Con qué herramientas construimos aplicaciones de red?
- **Propósito:** Explorar las tecnologías que permiten implementar aplicaciones distribuidas, desde el control fino de los sockets hasta las abstracciones de la Web.
- La tecnología elegida determina cuánto control tenemos y cuánta complejidad debemos manejar.
- **Preguntas importantes:**
 - ¿Qué exige cada tecnología del programador?
 - ¿Cómo elegir entre alto nivel (web) y bajo nivel (sockets)?
 - ¿Cómo impacta la tecnología en el diseño de la aplicación?

Bloque 3: Tecnologías de implementación

Sockets

- **Los sockets aparecen como una abstracción**
- Un socket es un punto de conexión que permite a una aplicación:
 - Enviar y recibir datos por la red
 - Sin **conocer los detalles internos** de TCP o UDP
 - Usando una **interfaz uniforme** en cualquier sistema operativo
- Un socket no es un protocolo: es una abstracción para acceder a protocolos de transporte.
 - Los sockets se usan para invocar a TCP o UDP a través del sistema operativo.

Bloque 3: Tecnologías de implementación

Sockets

- **Qué resuelven los sockets**
 - **Unifican:** la misma interfaz de programación de aplicaciones para TCP y UDP.
 - **Simplifican:** esconden la complejidad del transporte.
 - **Estandarizan:** permiten que programas escritos en distintos lenguajes se comuniquen igual.
 - **Aíslan:** la aplicación se enfoca en su lógica, no en la mecánica de la red.

Bloque 3: Tecnologías de implementación

Sockets

- **Socket API**: conjunto básico de funciones.
 - Permiten a dos programas comunicarse a través de la red enviando y recibiendo datos directamente sobre protocolos como TCP y UDP.
 - Obliga a manejar algunos **detalles de transporte** como puertos, direcciones, envío y recepción de datos.
- Los sockets existen para que las aplicaciones puedan comunicarse por la red **sin tener que implementar la red**.
- Son la frontera entre la lógica de aplicación y los mecanismos de transporte.

Bloque 3: Tecnologías de implementación

Sockets

Socket API

- API = conjunto básico de funciones
- Lo van a ver en el taller

Primitive	Meaning
SOCKET	Create a new communication endpoint
BIND	Associate a local address with a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Passively establish an incoming connection
CONNECT	Actively attempt to establish a connection
SEND	Send some data over the connection
RECEIVE	Receive some data from the connection
CLOSE	Release the connection

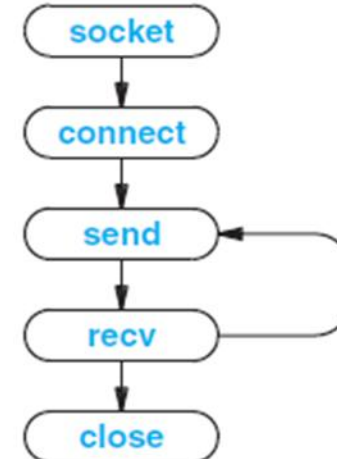
Figure 6-5. The socket primitives for TCP.

Para UDP no se usan LISTEN, ACCEPT y CONNECT.
Para UDP CLOSE se usa para cerrar un socket.

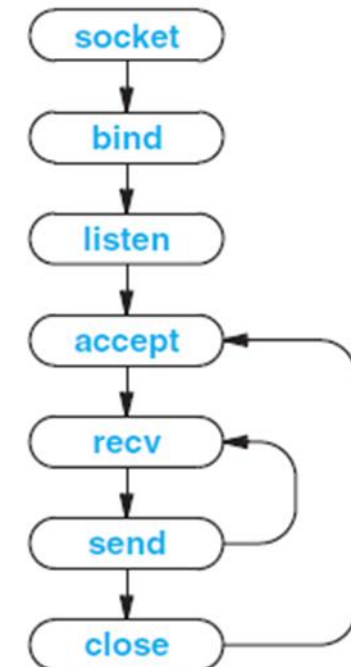
Bloque 3: Tecnologías de implementación Sockets

- **El Servidor TCP:** prepara un punto de encuentro.
 - El servidor crea un “lugar” donde los clientes pueden conectarse.
- 1. **Crear socket** (socket): Preparar un punto de comunicación.
- 2. **Asociar a un puerto** (bind): Elegir en qué puerto va a escuchar.
- 3. **Escuchar** (listen): Indicar que está listo para aceptar conexiones.
- 4. **Aceptar conexión** (accept): Cuando llega un cliente, se crea una conexión TCP independiente.
- 5. **Enviar / recibir datos**: Intercambio confiable y ordenado.
- 6. **Cerrar conexión**: Liberar recursos cuando termina la interacción.

CLIENT SIDE



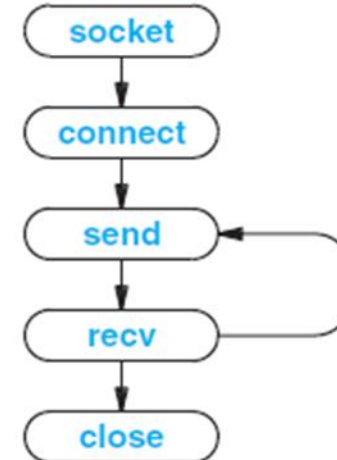
SERVER SIDE



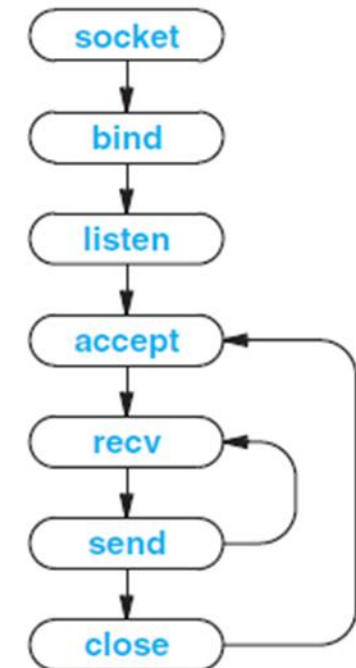
Bloque 3: Tecnologías de implementación Sockets

- **Cliente TCP:** inicia la comunicación.
 - El cliente busca al servidor y establece la conexión.
- 1. **Crear socket:** Preparar su punto de comunicación.
- 2. **Conectar** (connect): Iniciar la conexión TCP con la IP y puerto del servidor.
- 3. **Enviar / recibir datos:** Usar la conexión ya establecida para intercambiar mensajes.
- 4. **Cerrar conexión:** Indicar que terminó de usar el servicio.

CLIENT SIDE



SERVER SIDE



Bloque 3: Tecnologías de implementación

La Web

- Vimos que los sockets nos permiten construir aplicaciones distribuidas “desde abajo”, controlando directamente cómo se envían y reciben los mensajes entre dos computadoras.
 - Este enfoque nos muestra la esencia de la comunicación en red: abrir un canal, intercambiar datos y cerrar la conexión.
 - Pero justamente por ser tan básico, también nos exige encargarnos de muchos detalles: cómo estructurar los mensajes, cómo interpretar lo que llega, cómo manejar errores y cómo organizar la interacción entre cliente y servidor.
- A medida que las aplicaciones crecieron en complejidad, surgió la necesidad de contar con un nivel más alto de organización: un conjunto de reglas comunes que permitiera a cualquier cliente comunicarse con cualquier servidor sin tener que reinventar todo desde cero.
- Ese salto de abstracción es lo que da origen a **la Web**, donde protocolos como HTTP definen no solo cómo se envían los datos, sino también **qué significan y cómo deben comportarse los participantes**.
 - La Web se apoya en sockets, pero los envuelve en un marco mucho más estructurado, estandarizado y fácil de usar.
 - Pasamos así de construir la comunicación “a mano” a trabajar con un ecosistema que ya trae convenciones, formatos y comportamientos bien definidos.

Bloque 3: Tecnologías de implementación

La Web

- **Opciones para desarrollar aplicaciones de red:**
 - **La Web:**
 - ¿Tienen alguna idea de qué es la Web?

Bloque 3: Tecnologías de implementación

La Web

- **La Web:**
 - La Web es una plataforma de aplicaciones de red basada en protocolos estandarizados (como HTTP/HTTPS) que permite acceder a información y servicios desde cualquier navegador, sin manejar detalles de bajo nivel como sockets.
 - ¿Conocen alguna ventaja de usar la web?

Bloque 3: Tecnologías de implementación

La Web

- **La Web:**
 - La Web es una plataforma de alto nivel de aplicaciones de red basada en protocolos estandarizados (como HTTP/HTTPS) que permite acceder a información y servicios desde cualquier navegador, sin manejar detalles de bajo nivel como sockets.
 - **Ventajas de la Web**
 - Se trabaja con una Abstracción más alta al no tener que gestionar paquetes, puertos, ni protocolos de capa de transporte.
 - Cualquier dispositivo con navegador puede usar la aplicación sin instalar nada.
 - La web permite enfocarse en la lógica de la aplicación en lugar de la infraestructura de red.
 - Permite interoperabilidad entre lenguajes, sistemas y dispositivos.
 - Es la base de la mayoría de las aplicaciones modernas.

Bloque 3: Tecnologías de implementación

Comparación de tecnologías de implementación

criterio	Sockets	Web
Nivel de abstracción	Bajo: control directo sobre el envío y la recepción de datos.	Alto: el protocolo ya define estructura, significado y reglas.
Flexibilidad	Muy alta: se puede diseñar cualquier protocolo propio.	Media: se siguen reglas del protocolo HTTP y sus métodos.
Estructura de los mensajes	La define el programador desde cero.	Ya viene estandarizada (p.ej. con métodos, encabezados, cuerpo)
Facilidad de uso	Requiere más trabajo y cuidado con detalles.	Más simple: muchas tareas ya están resueltas.
Control sobre la comunicación	Total: tiempos, formato, orden flujo.	Parcial: HTTP decide gran parte del comportamiento.
Interoperabilidad	Limitada: ambos extremos deben implementar el mismo protocolo.	Muy alta: cualquier navegador puede comunicarse.
Costos de desarrollo	Más altos: hay que diseñar y mantener el protocolo.	Más bajo: usan estándares y herramientas existentes.
Seguridad	Depende de lo que programes.	HTTPS resuelve cifrado y autenticación básica.

Bloque 3: Tecnologías de implementación

- **Elegir Sockets cuando:**

- necesitas **máximo control** sobre la comunicación;
- quieres diseñar un **protocolo propio**;
- la aplicación requiere **baja latencia** o comunicación continua (juegos, chat, streaming);
- el formato de los mensajes debe ser **muy eficiente**.

- **Elegir Web (HTTP/HTTPS) cuando:**

- quieres **simplicidad** y evitar reinventar protocolos;
- necesitas **interoperabilidad** con navegadores, móviles o servicios externos;
- vas a crear un sitio web o una aplicación web;
- buscas **herramientas maduras**, seguridad integrada y facilidad de pruebas.

Bloque 3: Tecnologías de implementación

- **Resumen:**

- Analizamos cómo las tecnologías de implementación determinan el nivel de abstracción con el que trabajamos al construir aplicaciones de red.
 - La API de sockets ofrece control total sobre la comunicación, permitiendo gestionar puertos, direcciones y detalles del transporte, pero exige un esfuerzo mayor del programador.
 - En cambio, la Web proporciona una abstracción más alta que oculta muchos detalles y facilita la portabilidad, permitiendo que cualquier dispositivo con navegador acceda a la aplicación.
- Discutimos que cada tecnología implica compromisos: mayor control implica mayor complejidad, mientras que mayor abstracción implica ceder parte del control a la plataforma.
 - Esto afecta la flexibilidad, la velocidad de desarrollo y la portabilidad de la aplicación.
- Finalmente, vimos que la tecnología elegida influye directamente en el diseño: determina cuán explícito debe ser el protocolo, qué responsabilidades recaen en el programador y qué partes se delegan a la infraestructura.
- **Idea final:** la tecnología elegida define el equilibrio entre control y simplicidad.